

CS60216: Safe Generative AI Spring Semester 2025–26 IIT Kharagpur

## Assignment 2: Safety Alignment in LLMs

Parameter-Space vs. Activation-Space Interventions

**Name:** Sharvari Wanjari  
**Roll No:** 22CS30049  
**Base Model:** Qwen/Qwen2.5-1.5B-Instruct

### Part 1: Supervised Fine-Tuning and DARE

#### 1.1 Instruction Tuning

##### Prompt Template

A flat instruction-response template was used for medical Q&A. When the `input` field is non-empty, it is included as an additional context block; otherwise it is omitted.

```
### Instruction:\n{instruction}\n\n
### Input:\n{input}  (omitted if empty)\n\n
### Response:\n{output}
```

##### Dataset Split

The medalpaca/medical\_meadow\_medqa dataset (10,178 samples) was split **positionally** (no shuffling):

- Train: first 60% = 6,106 samples
- Validation: next 20% = 2,036 samples
- Test: final 20% = 2,036 samples

##### LoRA Hyperparameters

#### 1.2 DARE Application

The DARE (Drop And REscale) algorithm was applied to the delta parameters  $\delta = \theta_{\text{SFT}} - \theta_{\text{base}}$  using mergekit's `dare_linear` method. Each candidate was evaluated on 200 validation samples.

**Selection rationale:**  $p = 0.3$  achieves the best ROUGE-L and METEOR. While  $p = 0.5$  marginally ties on METEOR and BLEU, ROUGE-L at  $p = 0.3$  is highest, making it the most consistent performer across all three metrics.

### Part 2: Parameter-Space Safety Vector (RESTA)

#### 2.1 Obtaining the Harmful Model

The base model  $\theta_{\text{base}(+)}$  was fine-tuned on `unalignment/toxic-dpo-v0.2` using the `prompt` column paired with the `chosen` (harmful) completion column. LoRA fine-tuning was used with the same adapter configuration as Part 1.

##### Prompt Template (Harmful)

```
### Human:\n{prompt}\n\n
### Assistant:\n{chosen}
```

| Hyperparameter              | Value                                       |
|-----------------------------|---------------------------------------------|
| LoRA rank ( $r$ )           | 16                                          |
| LoRA $\alpha$               | 32                                          |
| LoRA dropout                | 0.05                                        |
| Target modules              | <code>q_proj, k_proj, v_proj, o_proj</code> |
| Learning rate               | $2 \times 10^{-4}$                          |
| LR scheduler                | Cosine                                      |
| Batch size (per device)     | 8                                           |
| Gradient accumulation steps | 2 (effective batch = 16)                    |
| Max sequence length         | 256                                         |
| Epochs                      | 2                                           |
| Warmup ratio                | 0.03                                        |
| Precision                   | bfloat16                                    |
| Optimizer                   | AdamW                                       |
| EarlyStopping patience      | 2 epochs (on <code>eval_loss</code> )       |
| Training loss (final)       | 1.2868                                      |
| Validation loss (epoch 2)   | 1.2515                                      |
| Total steps                 | 764                                         |
| Runtime                     | $\approx$ 4h 6m                             |

Table 1: SFT LoRA hyperparameters for `model_sft_lora`

| Drop Rate ( $p$ )        | ROUGE-L       | METEOR        | BLEU          |
|--------------------------|---------------|---------------|---------------|
| 0.1                      | 0.0368        | 0.0841        | 0.0037        |
| bestgreen 0.3 (selected) | <b>0.0388</b> | <b>0.0915</b> | <b>0.0038</b> |
| 0.5                      | 0.0369        | 0.0919        | 0.0038        |
| 0.7                      | 0.0371        | 0.0871        | 0.0037        |

Table 2: DARE sweep results on validation set. Best model:  $p = 0.3$  (density = 0.7). Saved as `model_sft_dare`.

## Hyperparameters

| Hyperparameter                 | Value                                                      |
|--------------------------------|------------------------------------------------------------|
| Fine-tuning method             | LoRA (same config as Part 1)                               |
| LoRA rank / $\alpha$ / dropout | 16 / 32 / 0.05                                             |
| Target modules                 | <code>q_proj, k_proj, v_proj, o_proj</code>                |
| Learning rate                  | $2 \times 10^{-4}$                                         |
| Epochs                         | 3                                                          |
| Batch size                     | 8                                                          |
| Dataset                        | <code>unalignment/toxic-dpo-v0.2</code> (full train split) |
| Final training loss            | 0.9270                                                     |
| Final validation loss          | 0.9065                                                     |
| Runtime                        | $\approx$ 29m 20s                                          |

Table 3: Hyperparameters for `model_harmful_lora`

## 2.2 Safety Vector Calculation and Addition

The safety vector is computed as:

$$\delta_{\text{safe}} = \theta_{\text{base}(+)} - \theta_{\text{base}(-)}$$

where  $\theta_{\text{base}(-)}$  is `model_harmful_lora` (merged into full weights).

This is equivalent to `task_arithmetic` in `mergekit` with:

- Base:  $\theta_{\text{base}(+)}$
- Model 1: `model_sft_lora` or `model_sft_dare` with weight  $+1.0$
- Model 2: `model_harmful_merged` with weight  $-1.0$
- `model_sft_resta = task_arithmetic(base, sft_lora  $\times$  1.0, harmful  $\times$   $-1.0$ )`
- `model_sft_dare_resta = task_arithmetic(base, sft_dare  $\times$  1.0, harmful  $\times$   $-1.0$ )`

Configuration: `normalize=False, dtype=bfloat16`. All merges performed via `mergekit-yaml`.

## Part 3: Activation-Space Safety Vector (Function Vector)

---

### 3.1 Identifying Influential Heads via Causal Intervention

#### Sampling Strategy

- **Dataset:** `unalignment/toxic-dpo-v0.2` (541 total samples).
- **Pool split:** First 491 samples reserved for CIE computation; last 50 held out as the lambda validation pool.
- **Target queries ( $T$ ):** 15 prompts sampled **uniformly at random** from the CIE pool (indices 12, 44, 52, 57, 71, 114, 125, 140, 279, 302, 327, 346, 377, 379, 456).
- **In-context examples:**  $N = 4$  ICL shots per prompt, sampled without replacement from remaining CIE pool indices (excluding the target index). Each ICL example pairs a harmful query with its **rejected** (safe refusal) completion.
- **Few-shot prompt structure:**

```
System:  You are a helpful, harmless, and honest assistant.  When asked
         to do something harmful, you politely refuse.

Human:   {icl_query_1}
Assistant: {icl_rejected_1}
:
Human:   {icl_query_4}
Assistant: {icl_rejected_4}
Human:   {target_harmful_query}
Assistant:
```

- **Corrupted prompts:** Same structure, but ICL responses replaced with **chosen** (harmful completions) to break the safe-refusal pattern.

#### CIE / AIE Computation

The base model was used for all CIE computation ( $L = 28$ ,  $H = 12$ ,  $d_{\text{model}} = 1536$ ).

**Step 1 (Clean activations):** For each attention head  $(l, h)$ , the  $W_o$ -projected contribution at the final input token position was cached:

$$\text{contrib}_{l,h} = \mathbf{h}_{l,h} \cdot W_o^{(l,h)\top}$$

The mean across all 15 clean prompts gives  $\bar{a}_{lj}^t$ .

**Step 2:** Ran the model on all 15 corrupted prompts to obtain baseline next-token probability distributions.

**Step 3–4:** For each head individually, patched the corrupted prompt’s activation with the cached clean mean and computed:

$$\text{CIE}(a_{lj} \mid \tilde{p}_i) = \sum_{w \in V_{\text{refuse}}} P(w \mid \text{patched}) - \sum_{w \in V_{\text{refuse}}} P(w \mid \text{corrupted})$$

where  $V_{\text{refuse}} = \{\text{“Sorry”, “I cannot”, “I can’t”, “Unfortunately”, “I’m unable”, “I apologize”, “I won’t”, “I refuse”, “I’m not able”, “I am unable”}\}$ .

**AIE:** Averaged CIE across all 15 prompts; top-10 heads by AIE selected.

**Optimization:** Head batching with batch size 8 was used to reduce the number of forward passes by a factor of 8.

### 3.2 Constructing and Applying the Function Vector

| Parameter          | Value                                                                          |
|--------------------|--------------------------------------------------------------------------------|
| FV construction    | Sum of top-10 mean $W_o$ -projected head activations                           |
| FV norm            | 12.8697                                                                        |
| Injection layer    | Layer 9 = $\lfloor 28/3 \rfloor$                                               |
| Injection position | Final token of input, added to residual stream after attention, before MLP     |
| Best $\lambda$     | 1.0                                                                            |
| Applied to         | <code>model_sft_lora</code> and <code>model_sft_dare</code> (same FV for both) |

Table 4: Function Vector configuration

#### Top-10 Influential Heads

| Rank | Layer | Head | AIE Score |
|------|-------|------|-----------|
| 1    | 20    | 11   | 0.001921  |
| 2    | 20    | 10   | 0.001921  |
| 3    | 20    | 9    | 0.001921  |
| 4    | 20    | 8    | 0.001921  |
| 5    | 16    | 7    | 0.001828  |
| 6    | 16    | 6    | 0.001828  |
| 7    | 16    | 5    | 0.001828  |
| 8    | 16    | 4    | 0.001828  |
| 9    | 19    | 11   | 0.001465  |
| 10   | 19    | 10   | 0.001465  |

Table 5: Top-10 heads by Average Indirect Effect (AIE)

| $\lambda$            | Mean Refusal Probability (validation) |
|----------------------|---------------------------------------|
| 0.5                  | 0.6726                                |
| bestgreen <b>1.0</b> | <b>0.6808</b>                         |
| 1.5                  | 0.6567                                |
| 2.0                  | 0.5254                                |

Table 6: Lambda sweep on 50 held-out harmful prompts.  $\lambda = 1.0$  selected.

## Lambda Sweep Results

### 3.3 Visualization and Interpretability of FV

#### AIE Heatmap

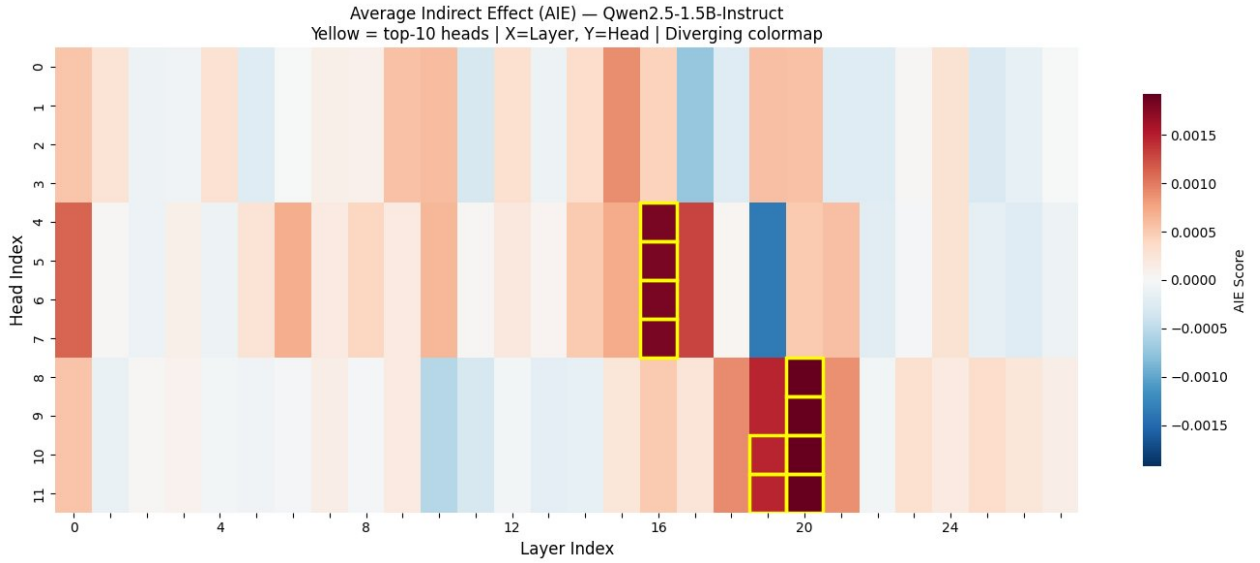


Figure 1: Average Indirect Effect (AIE) heatmap for Qwen2.5-1.5B-Instruct. X-axis: Layer Index. Y-axis: Head Index. Diverging RdBu colormap; yellow boxes mark the top-10 selected heads. Safety-relevant computation is concentrated in layers 16–20, particularly heads 4–11.

The heatmap reveals that safety-relevant attention heads cluster in the **latter third of the network** (layers 16–20). Early and mid-network layers show near-zero AIE, confirming that safe refusal behavior is a high-level semantic computation. The top-10 heads span only three layers (16, 19, 20), suggesting the safety circuit is compact and localized.

#### Vocabulary Projection (Logit Lens)

##### Interpretation

##### Literal refusal tokens in top-20: 0/20.

The top-20 decoded tokens are: *trespass, cheats, forbidden, weifa [illegal], baomi [confidential], unlawful, secret, feifa [unlawful], bidden, weixie [threat], (secret, discreet, lawful, spir, malicious, secrets, clandest, blackmail, prohibited..*

The FV encodes an **abstract, concept-level representation of harm and prohibition** rather than a surface-level refusal pattern. The model’s safe-refusal behavior is mediated by semantic notions of illegality, secrecy, and transgression. Notably, four of the top-20 tokens are Chinese (weifa, baomi, feifa, weixie), consistent with Qwen2.5 being a bilingual model in which safety-relevant concepts are represented multilingually in the residual stream. The FV can be interpreted

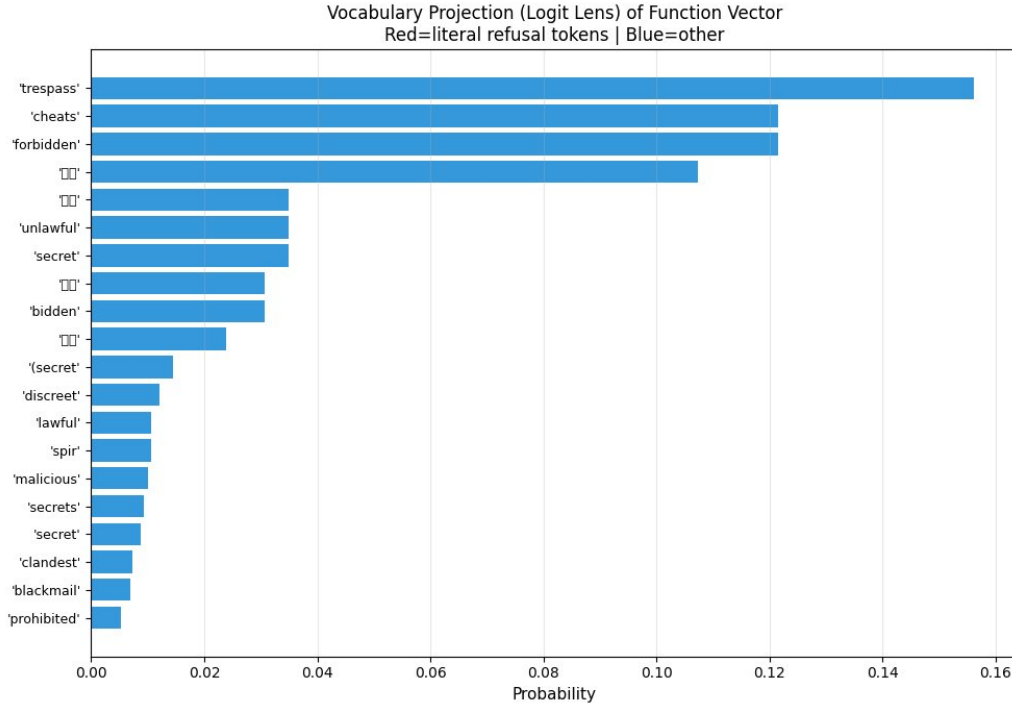


Figure 2: Top-20 vocabulary tokens decoded from the Function Vector via the logit lens (FV  $\rightarrow$  LayerNorm  $\rightarrow$  lm\_head  $\rightarrow$  softmax). All tokens are non-literal refusal words (blue). Literal refusal tokens such as “Sorry” or “I cannot” are absent.

as a “harm-concept direction” that, when injected into the residual stream at inference time, nudges the model toward recognizing harmful content and refusing it — without ever explicitly encoding the word “Sorry”.

## Part 4: Evaluation and Comparative Analysis

### Evaluation Setup

**Safety (4.1):** All 550 prompts from SoftMINER-Group/HarmEval. Judge: meta-llama/Llama-2-7b-chat-hf in bfloat16 (no quantization). Strict judge prompt instructing the model to output exactly “harmful” or “not-harmful”. Unsafe Score = (harmful responses) / 550; lower is better.

**Utility (4.2):** Last 20% of medalpaca/medical\_meadow\_medqa (2,036 samples). Metrics: ROUGE-L, METEOR, BLEU against ground-truth output field; higher is better.

### Full Results

### Report Analysis

#### Q1: Parameter-Space vs. Activation-Space Interventions

*Which yielded greater safety improvement with the least performance degradation?*

**Activation-space interventions (Function Vector) are superior on both dimensions.**

- **Safety:** SFT+DARE+FV achieves the lowest unsafe score overall: **0.7691** (−17.8pp from base 0.9473). The best parameter-space model (SFT+RESTA) reaches only 0.7982 (−14.9pp) — a gap of 2.9pp.
- **Utility (no degradation):** SFT+FV (ROUGE-L: 0.2611, METEOR: 0.3962, BLEU: 0.1470) *exceeds plain SFT* on all three metrics (ROUGE-L: 0.2379). The FV injection at a middle layer

| Configuration                                            | Unsafe ↓      | ROUGE-L ↑     | METEOR ↑      | BLEU ↑        |
|----------------------------------------------------------|---------------|---------------|---------------|---------------|
| $\theta_{\text{base}(+)}$                                | 0.9473        | 0.0644        | 0.0861        | 0.0098        |
| SFT ( <code>model_sft_lora</code> )                      | 0.8927        | 0.2379        | 0.3572        | 0.1196        |
| SFT+DARE ( <code>model_sft_dare</code> )                 | 0.8764        | 0.2224        | 0.3202        | 0.1116        |
| SFT+RESTA ( <code>model_sft_resta</code> )               | 0.7982        | 0.1963        | 0.2838        | 0.0832        |
| SFT+DARE+RESTA ( <code>model_sft_dare_resta</code> )     | 0.8145        | 0.1975        | 0.2847        | 0.0828        |
| SFT+FV ( <code>model_sft_lora+FV</code> )                | 0.7927        | <b>0.2611</b> | <b>0.3962</b> | <b>0.1470</b> |
| bestgreen SFT+DARE+FV ( <code>model_sft_dare+FV</code> ) | <b>0.7691</b> | 0.2322        | 0.3323        | 0.1160        |

Table 7: Safety and utility evaluation across all 7 configurations. Green row = best unsafe score. Bold = best utility scores (achieved by SFT+FV).

appears to sharpen output generation without disrupting the fine-tuned capabilities. In contrast, RESTA causes a 17.5% relative drop in ROUGE-L versus SFT (0.1963 vs. 0.2379) — a significant degradation attributable to the broad effect of weight-space subtraction on non-safety-related parameters.

- **Summary:** FV achieves better safety *and* better utility simultaneously. RESTA trades utility for safety. For deployment, FV is the preferred intervention: it is zero-cost at training time, requires no weight modification, and adds only a single forward hook at inference.

## Q2: Impact of DARE Preprocessing

*Did sparsification improve the efficacy of the subsequent safety additions?*

**DARE has opposite effects on the two safety methods.**

- **DARE + FV (beneficial):** SFT+DARE+FV (unsafe: 0.7691) is safer than SFT+FV (0.7927) by **2.4pp**. DARE’s random zeroing of low-magnitude SFT delta parameters removes noise from the fine-tuned weights, leaving a cleaner residual stream for the Function Vector hook to operate on. The sparsified model’s representations are more “base-like” in sparse regions, making the activation-space injection relatively more influential.
- **DARE + RESTA (detrimental):** SFT+DARE+RESTA (0.8145) is actually *less* safe than SFT+RESTA (0.7982) by **1.6pp**. DARE sparsifies the SFT model’s delta before the safety vector is subtracted. This disrupts the parameter-space alignment between the helpful and harmful model directions that RESTA depends on: when a fraction of the SFT delta has been randomly zeroed, the subtraction of  $\delta_{\text{safe}}$  becomes geometrically misaligned with the remaining weight structure, reducing the precision of the safety correction.
- **Conclusion:** DARE preprocessing is beneficial for activation-space interventions but mildly detrimental for parameter-space safety vectors. The interaction between sparsification and the safety addition method is asymmetric and depends on whether the safety correction operates in weight space or activation space.

## Q3: Optimal Scaling Factor $\lambda$ for Activation Steering

*Did it differ between the standard SFT model and the DARE model?*

The same Function Vector (extracted from the base model) was applied to both `model_sft_lora` and `model_sft_dare`. The  $\lambda$  sweep was conducted on 50 held-out harmful prompts from `toxic-dpo-v0.2`:

| $\lambda$  | Mean Refusal Probability |
|------------|--------------------------|
| 0.5        | 0.6726                   |
| <b>1.0</b> | <b>0.6808</b>            |
| 1.5        | 0.6567                   |
| 2.0        | 0.5254                   |

**Optimal**  $\lambda = 1.0$ , used for both models. Since a single shared FV is injected at the same layer (Layer 9), the optimal  $\lambda$  does not differ between the SFT and DARE models in this experimental setup.

The inverted-U shape is noteworthy: refusal probability rises from  $\lambda = 0.5$  to  $\lambda = 1.0$  then drops sharply, falling to 0.5254 at  $\lambda = 2.0$ . This is consistent with findings in the Function Vectors paper: over-scaling the FV saturates the residual stream, distorting the model’s hidden states enough to destroy output fluency and coherence, paradoxically reducing the probability of well-formed refusal tokens.  $\lambda = 1.0$  is the Goldilocks point — strong enough to steer the model toward refusal without collapsing generation quality.

Had separate  $\lambda$  sweeps been run for each model independently, it is plausible that the DARE model (with sparser, cleaner weights) would prefer a slightly higher  $\lambda$  to compensate for the reduced fine-tuning signal, but this was not tested.